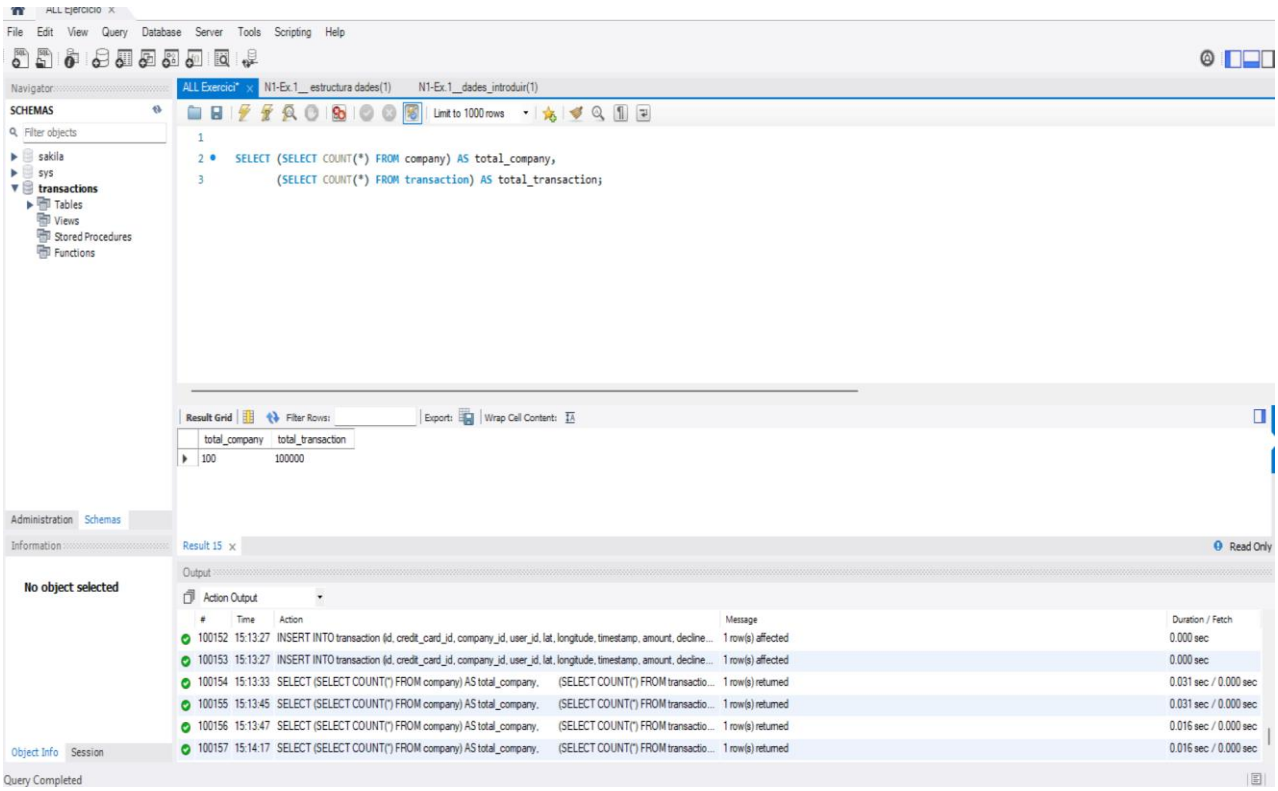
Exercici 1

Los documentos proporcionados definen la base de datos transactions y cargan dos tablas: company y transaction. El archivo de estructura crea las tablas; el archivo de datos las rellena mediante sentencias INSERT. En el fichero de datos hay 100 registros de empresas y 100.000 registros de transacciones.

Orden de ejecución

1) Ejecutar primero el archivo de estructura. 2) Ejecutar despues el archivo de insercion de datos. El

segundo archivo selecciona la base de datos con **USE transactions;**.



```sql
SELECT (SELECT COUNT(*) FROM company) AS total_company,
       (SELECT COUNT(*) FROM transaction) AS total_transaction;

SELECT * FROM company;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings

```sql
SELECT (SELECT COUNT(*) FROM company) AS total_company,
       (SELECT COUNT(*) FROM transaction) AS total_transaction;

SELECT * FROM transaction;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

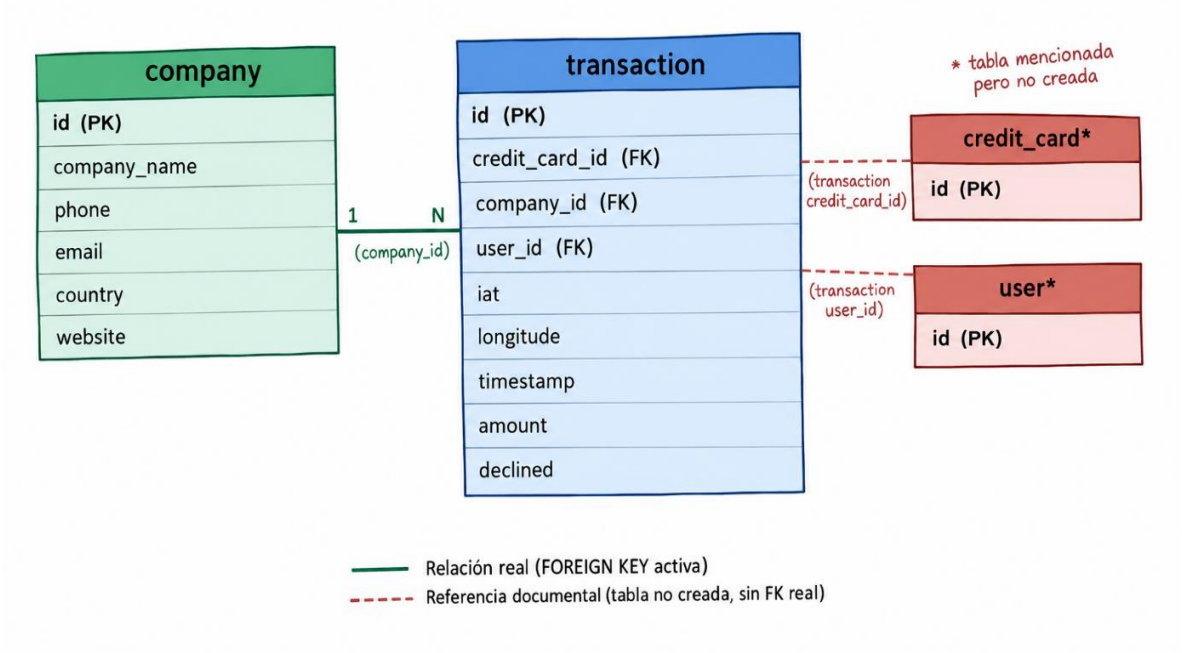
id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0

Result 17 | transaction 18 | Apply

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100162	15:22:49	SELECT * FROM comoany LIMIT 0, 1000	Error Code: 1146. Table 'transactions.comoany' doesn't exist	0.000 sec
100163	15:23:43	SELECT * FROM company LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
100164	15:26:01	SELECT * FROM transactions LIMIT 0, 1000	Error Code: 1146. Table 'transactions.transactions' doesn't exist	0.000 sec
100165	15:26:03	SELECT * FROM transactions LIMIT 0, 1000	Error Code: 1146. Table 'transactions.transactions' doesn't exist	0.000 sec
100166	15:26:57	SELECT (SELECT COUNT(*) FROM company) AS total_company, (SELECT COUNT(*) FROM transactio...	1 row(s) returned	0.032 sec / 0.000 sec
100167	15:26:57	SELECT * FROM transaction LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec



Tablas y variables
Las claves primarias identifican de forma unica cada fila. Los tipos indicados son los declarados en el SQL proporcionado.

Tabla company

Columna	Tipo de dato	Descripción
id	VARCHAR(15)	Clave primaria (PK) — identificador único de la empresa
company_name	VARCHAR(255)	Nombre de la empresa
phone	VARCHAR(15)	Teléfono de contacto
email	VARCHAR(100)	Dirección de correo electrónico
country	VARCHAR(100)	País donde opera la empresa
website	VARCHAR(255)	Sitio web de la empresa

Tabla transaction

Columna	Tipo de dato	Descripción
id	VARCHAR(255)	Clave primaria (PK) — identificador único de la transacción
credit_card_id	VARCHAR(15)	Referencia a la tarjeta de crédito utilizada (sin FK real)
company_id	VARCHAR(20)	Clave foránea (FK) real hacia company(id)
user_id	INT	Referencia al usuario que realiza la transacción (sin FK real)
lat	FLOAT	Latitud de la ubicación de la transacción
longitude	FLOAT	Longitud de la ubicación de la transacción
timestamp	TIMESTAMP	Fecha y hora de la transacción
amount	DECIMAL(10,2)	Importe de la transacción
declined	BOOLEAN	Indica si la transacción fue rechazada (1) o no (0)

Características principales y observaciones

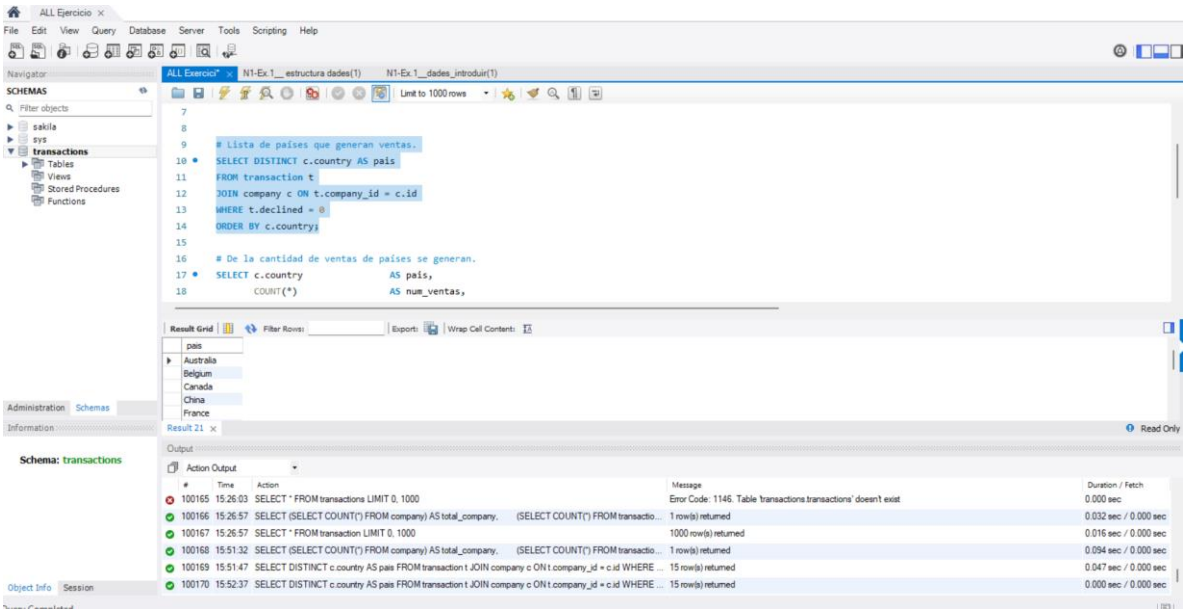
• Modelo relacional: los datos se organizan en tablas y se conectan mediante identificadores.
• Identificacion: company.id e transaction.id son las claves primarias.
• Relacion principal: transaction.company_id referencia company.id; una empresa puede aparecer en muchas transacciones.
• Datos numericos y temporales: el importe usa DECIMAL(10,2); las coordenadas son FLOAT y la fecha/hora es TIMESTAMP.

• Estado: declined es un valor booleano que registra si se rechazo la operacion.

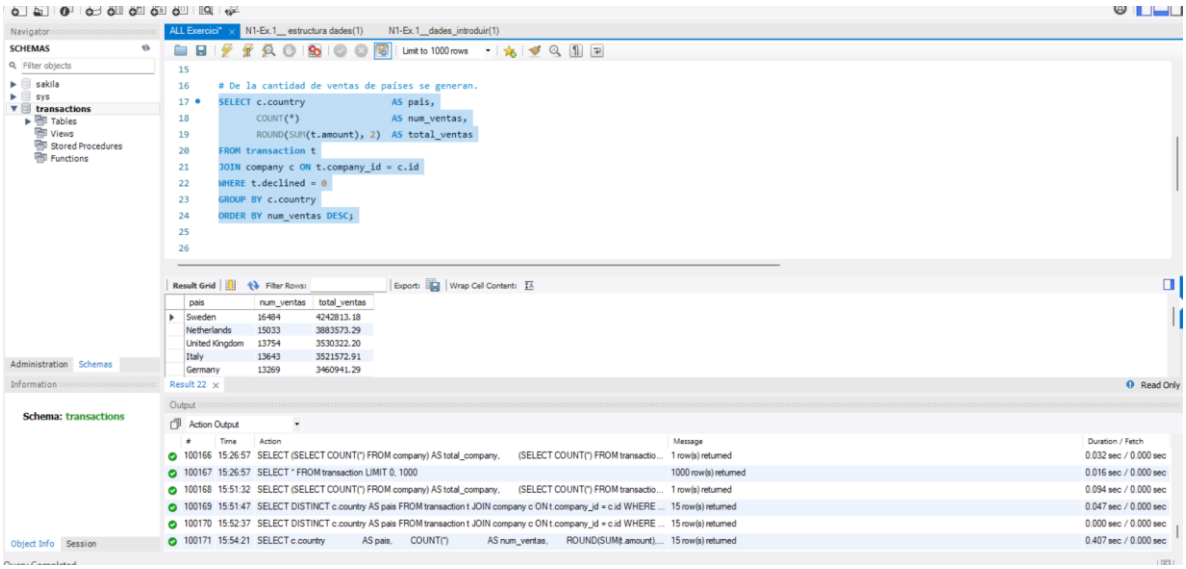
Exercici 2

Considero “venta” una transacción que no ha sido rechazada. Por eso, las consultas filtran declined = FALSE. La unión se realiza entre company.id y transaction.company_id.

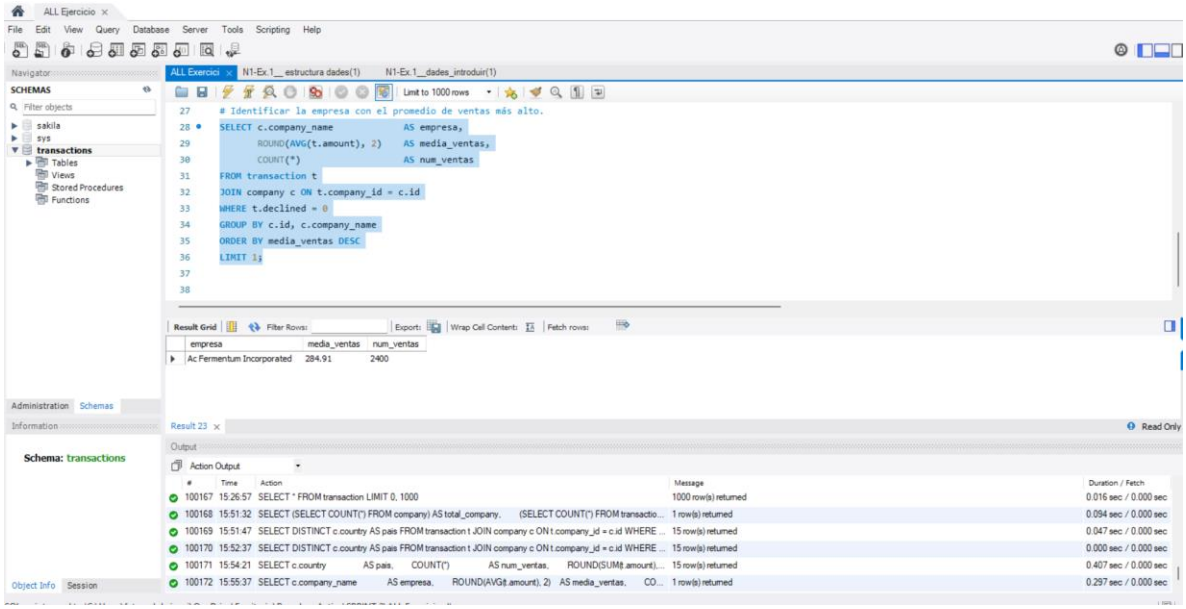
Países que generan ventas



Número de países desde los que se generan ventas



Compañía con la media de ventas más alta



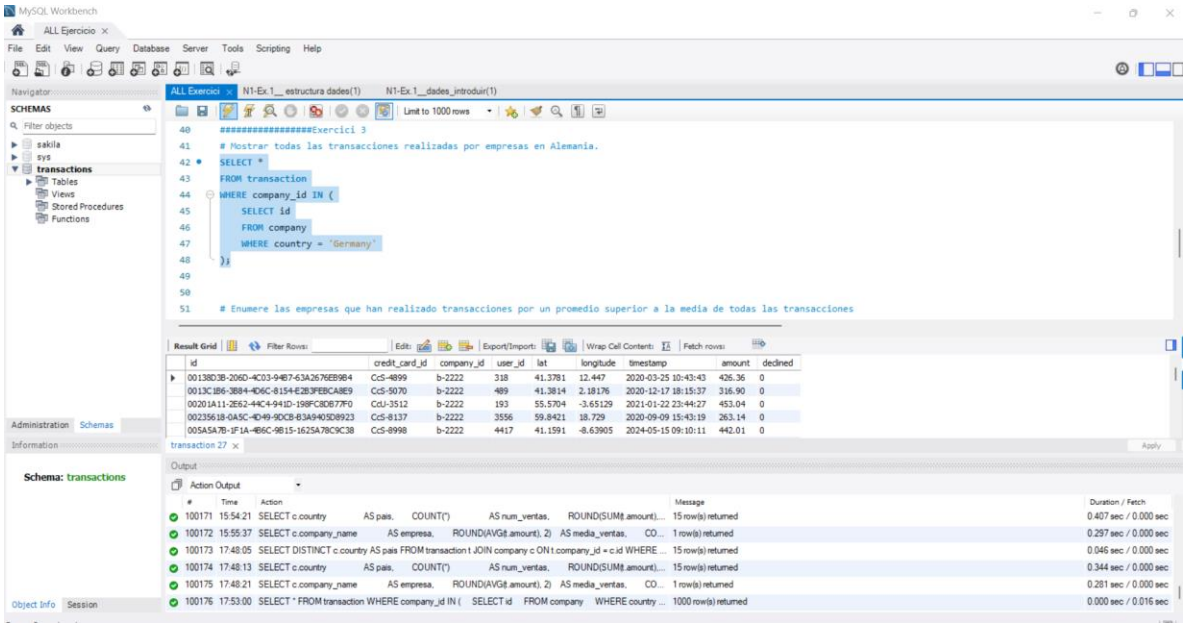
Resumen del cálculo

Los archivos incluyen 100 empresas y 100.000 transacciones. De ellas, 99.763 están aprobadas y 237 rechazadas. Al excluir las rechazadas, se obtienen 15 países con ventas. La compañía con el promedio más alto es Ac Fermentum Incorporated: 683.787,20 ÷ 2.400 = 284,9113, que se redondea a 284,91

Exercici 3

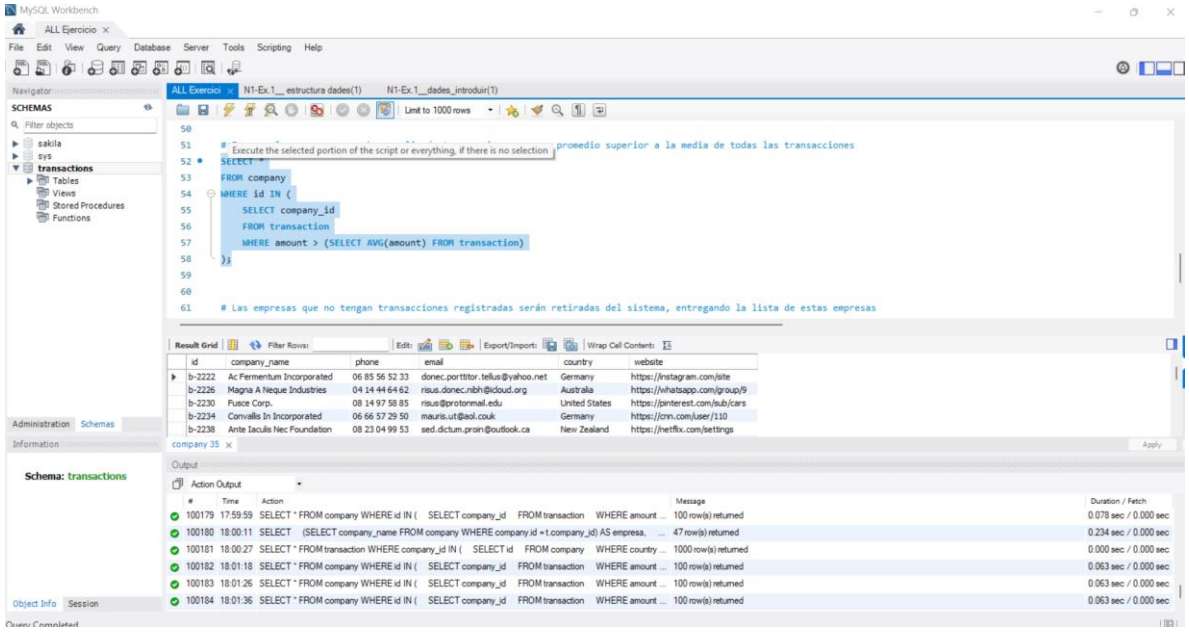
1. Transacciones de empresas de Alemania

La subconsulta obtiene los identificadores de las empresas cuyo país es Alemania. La consulta exterior muestra todas sus transacciones, sin excluir las rechazadas.



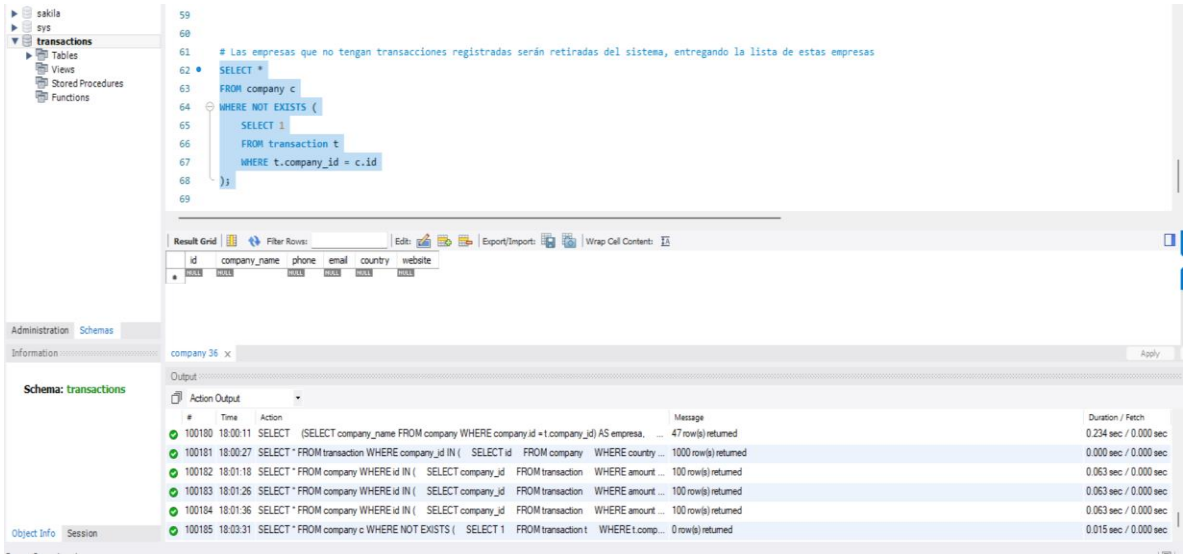
2. Empresas con alguna transacción superior a la media global

La subconsulta más interna calcula la media de amount para las 100.000 transacciones. La subconsulta exterior selecciona las empresas que tienen al menos una transacción por encima de esa media.



3. Empresas sin transacciones registradas

Esta consulta usa NOT EXISTS como subconsulta correlacionada para localizar empresas que no aparecen en la tabla de transacciones.



Exercici 4

1. Estructura propuesta

El archivo adjunto contiene 5.000 tarjetas con identificadores unicos y fechas de caducidad en formato MM/DD/YY. La tabla almacena los numeros como texto para conservar espacios o ceros iniciales; la fecha se convierte a DATE despues de importar.

CREATE TABLE credit_card (

id VARCHAR(15) NOT NULL PRIMARY KEY,

iban VARCHAR(34) NOT NULL,

pan VARCHAR(19) NOT NULL,

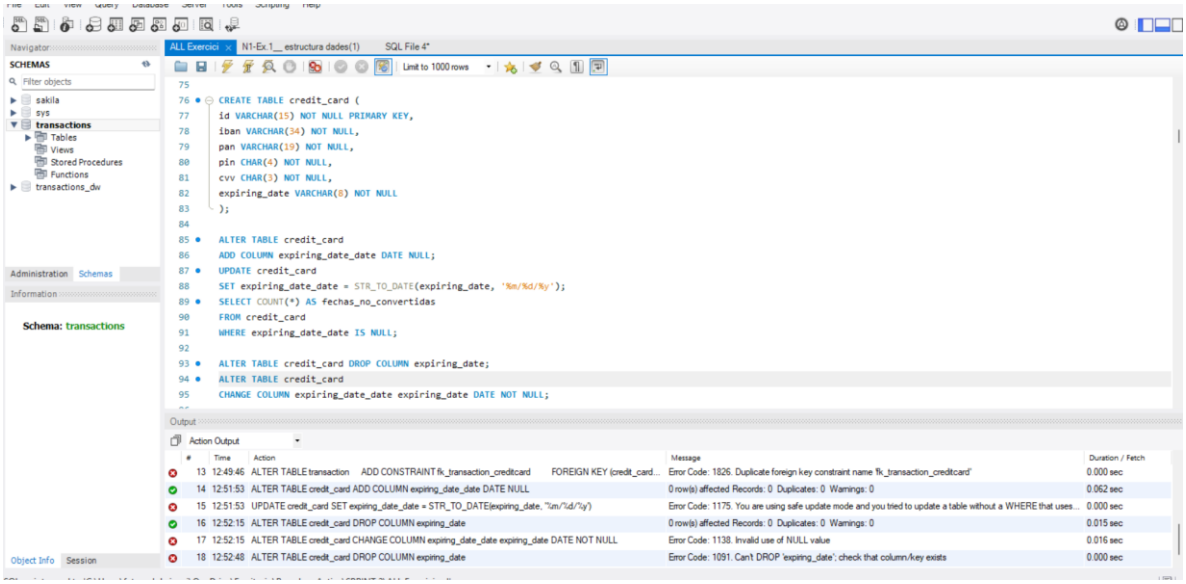
pin CHAR(4) NOT NULL,

cvv CHAR(3) NOT NULL,

expiring_date VARCHAR(8) NOT NULL

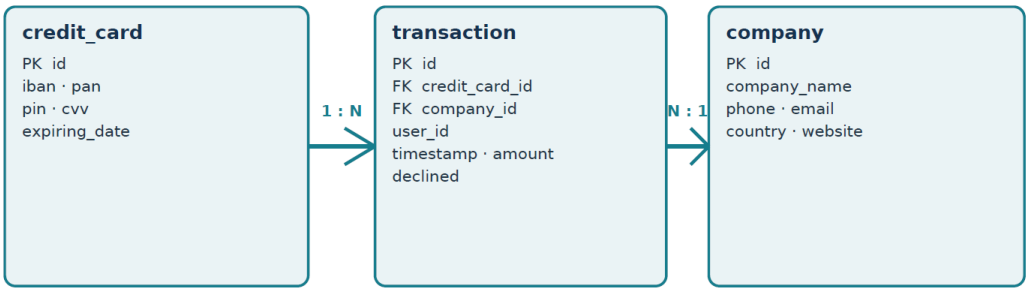
);

2-datos_introduir_credit(1).sql, que inserta las 5.000 filas en credit_card. Tras comprobar la carga, convierte la fecha textual a DATE



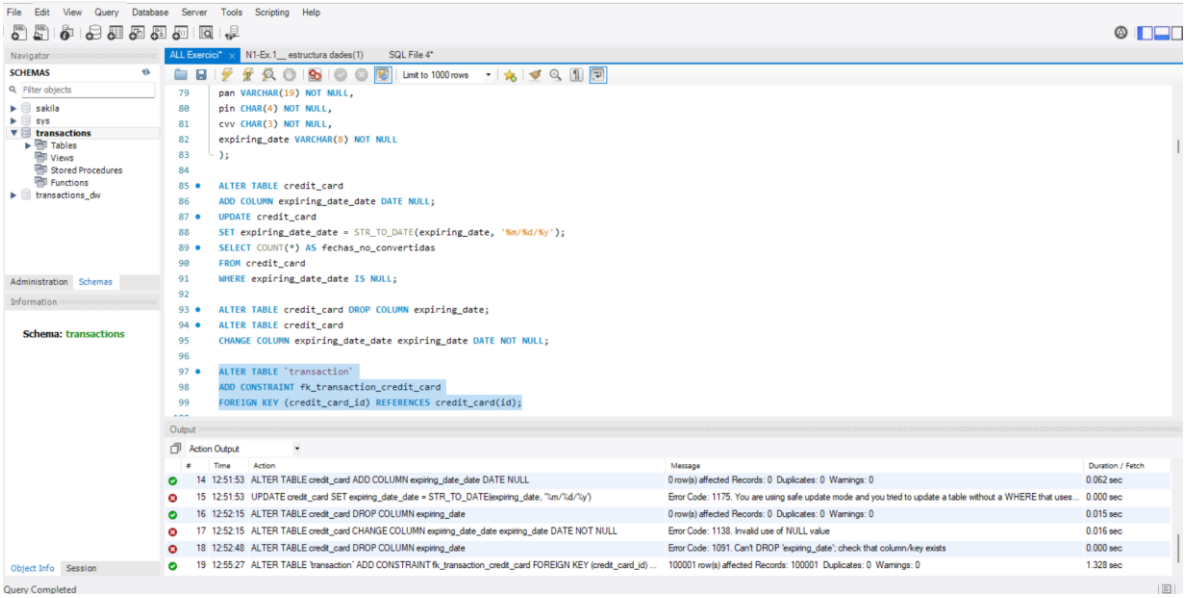
3. Relaciones y diagrama

La tarjeta y la empresa se relacionan a traves de la tabla transaction. Esta conserva que tarjeta se uso y en que empresa se realizo cada operacion.



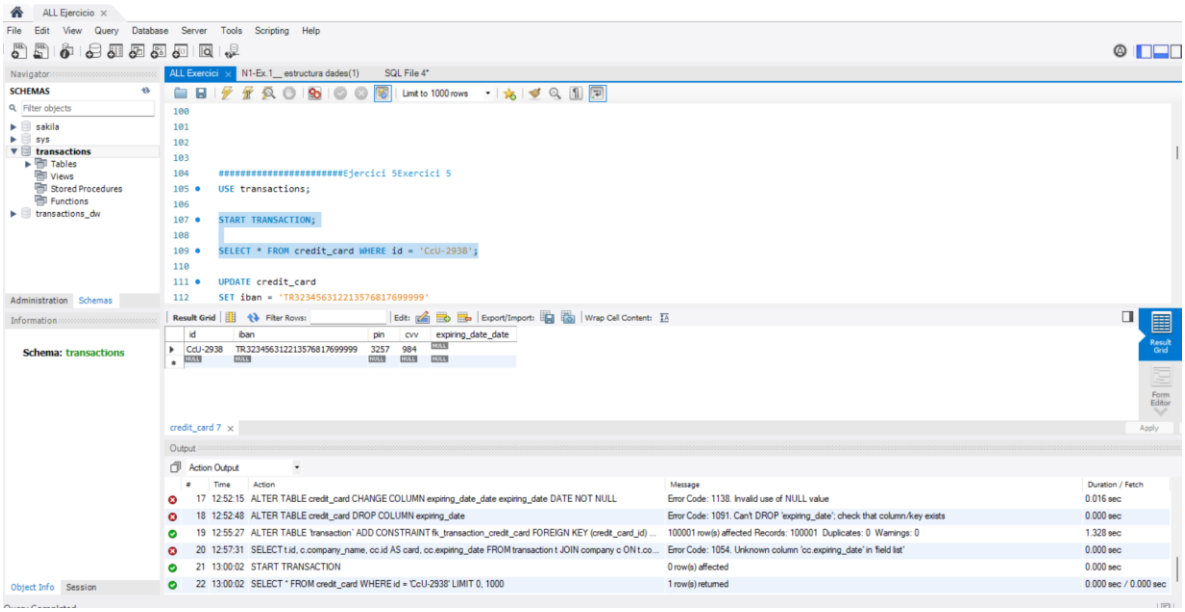
Descripción breve

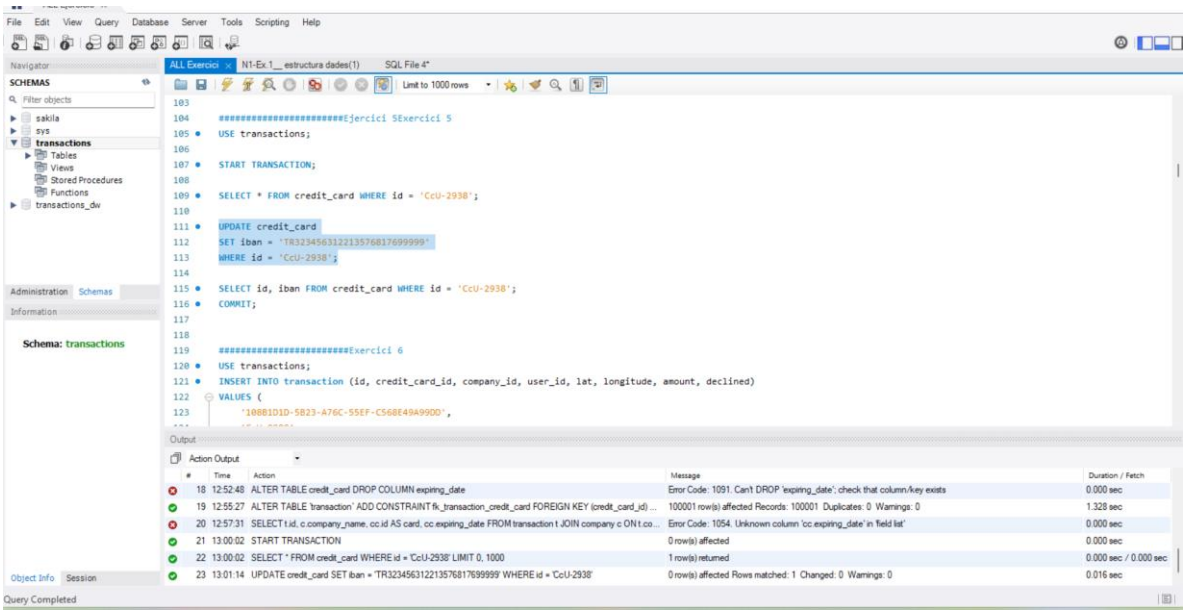
credit_card (1) → transaction (N): una tarjeta puede aparecer en muchas transacciones; cada transaccion referencia una tarjeta mediante credit_card_id. company (1) → transaction (N): una empresa puede tener muchas transacciones; cada una referencia la empresa mediante company_id. Asi, una tarjeta puede utilizarse en distintas empresas y una empresa puede recibir pagos de distintas tarjetas. No hace falta anadir una relacion directa entre credit_card y company.

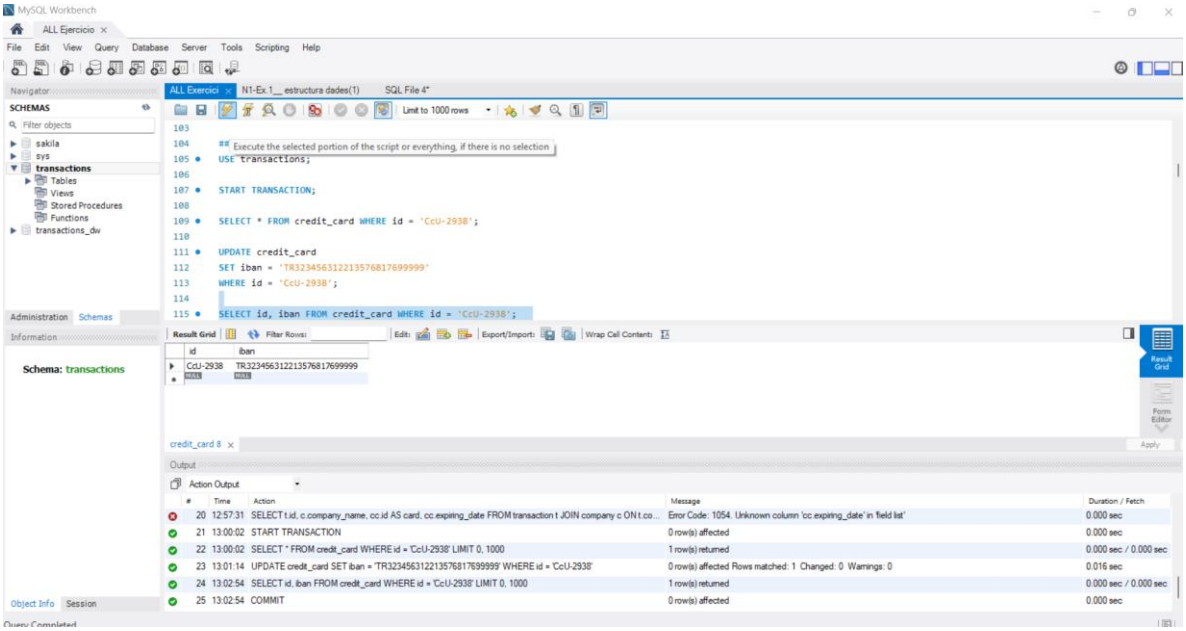


Exercici 5

Tarjeta: CcU-2938
Nuevo IBAN: TR323456312213576817699999



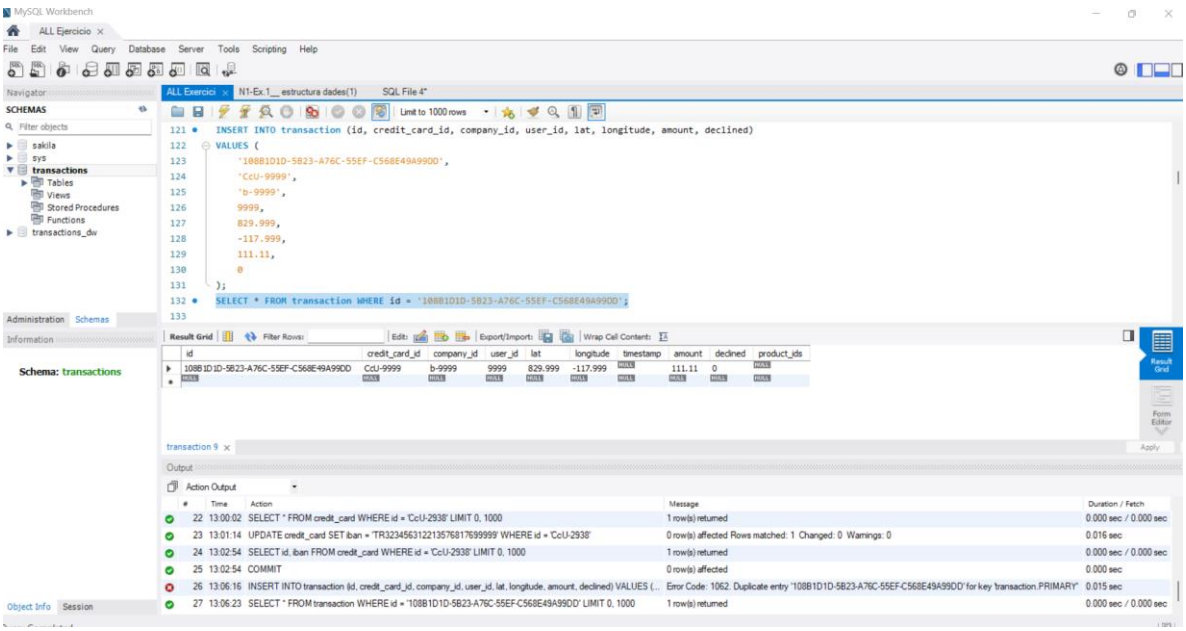



MySQL Workbench
ALL Ejercicio
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
sakila
sys
transactions
Tables
Views
Stored Procedures
Functions
transactions_dw
Administration Schemas
Information
Schema: transactions
Object Info Session
ALL Exercici
N1-Ex.1__estructura dades(1)
SQL File 4*
Limit to 1000 rows
Execute the selected portion of the script or everything, if there is no selection
USE transactions;
START TRANSACTION;
SELECT * FROM credit_card WHERE id = 'CcU-2938';
UPDATE credit_card
SET iban = 'TR323456312213576817699999'
WHERE id = 'CcU-2938';
SELECT id, iban FROM credit_card WHERE id = 'CcU-2938';
Result Grid
Filter Rows:
Edit:
Export/Import:
Wrap Cell Content:
id
iban
CcU-2938
TR323456312213576817699999
credit_card 8
Apply
Output
Action Output
Time Action Message Duration / Fetch
20 12:57:31 SELECT t.id, c.company_name, cc.id AS card, cc.expiring_date FROM transaction t JOIN company c ON t.co... Error Code: 1054. Unknown column 'cc.expiring_date' in 'field list' 0.000 sec
21 13:00:02 START TRANSACTION 0 row(s) affected 0.000 sec
22 13:00:02 SELECT * FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
23 13:01:14 UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938' 0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0 0.016 sec
24 13:02:54 SELECT id, iban FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
25 13:02:54 COMMIT 0 row(s) affected 0.000 sec
Query Completed

Exercici 6

Sentencia INSERT

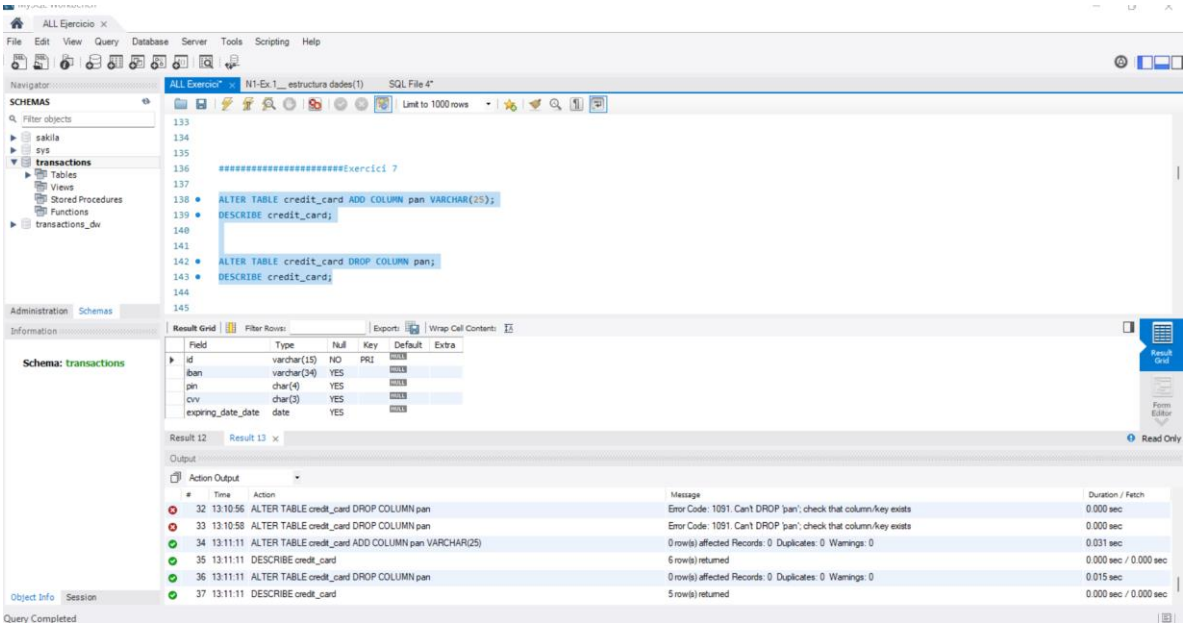
El siguiente SQL incorpora los campos que aparecen en la imagen. No se incluye timestamp porque no se proporciona en el enunciado; al omitirse, la base de datos aplicará su valor predeterminado (o NULL si la columna lo permite).



Exercici 7

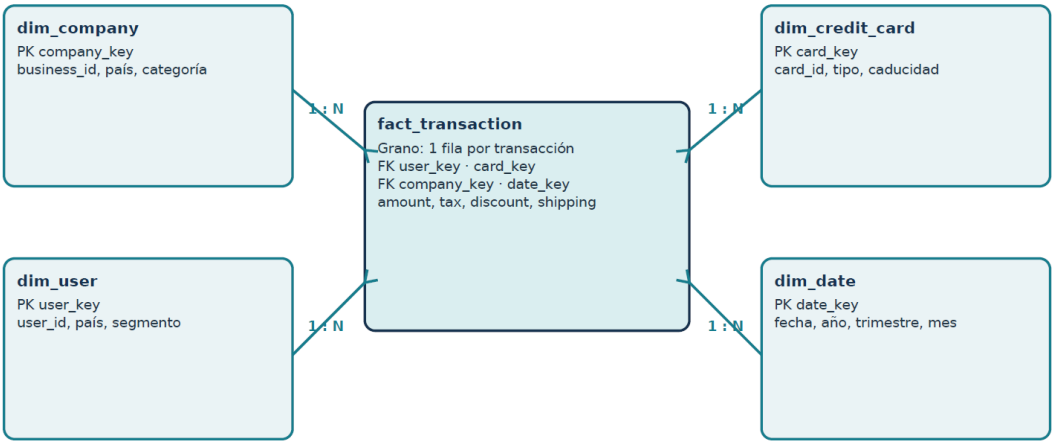
Se elimina la columna pan de la tabla credit_card

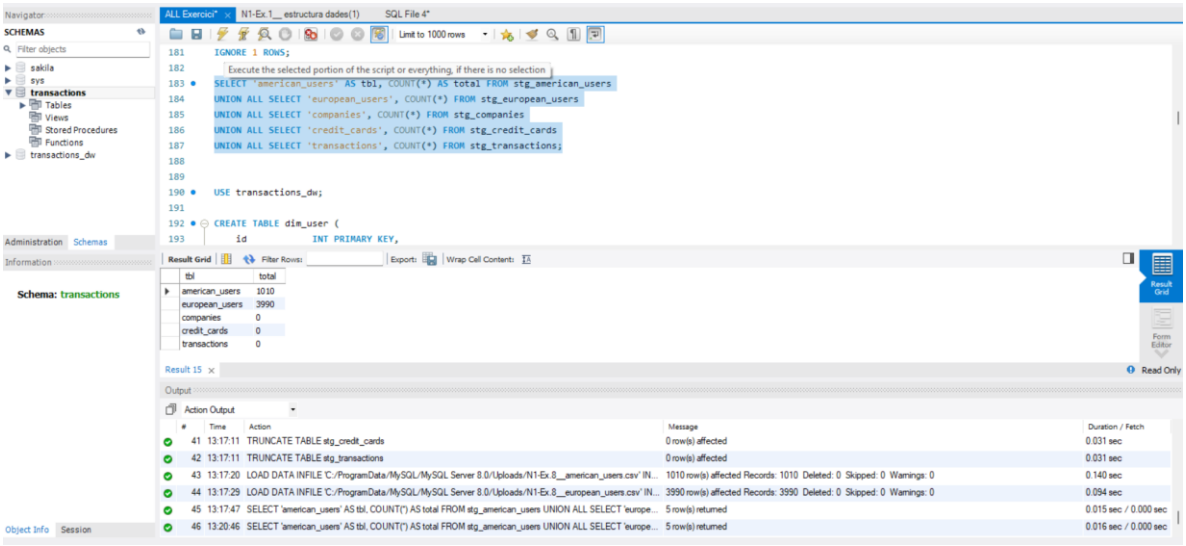
Después de ejecutar la modificación, DESCRIBE para mostrar la estructura actualizada



Exercici 8

Propuesta del modelo
Propongo un modelo con una tabla de hechos y cuatro dimensiones (cinco tablas en total). El grano de la tabla central es una fila por transacción. Así se podrán analizar importes, descuentos, impuestos y rechazos por empresa, usuario, tarjeta y fecha.





Preparación y carga

1. Combinar american_users.csv y european_users.csv en dim_user, añadiendo source_region para conservar la
procedencia.
2. Cargar companies.csv en dim_company y credit_cards.csv en dim_credit_card; convertir expiring_date al formato DATE.
3. Generar dim_date para las fechas incluidas en timestamp.
4. Cargar transactions.csv en fact_transaction resolviendo user_key, card_key y company_key a partir de sus claves de negocio; derivar date_key y conservar timestamp completo en transaction_ts.

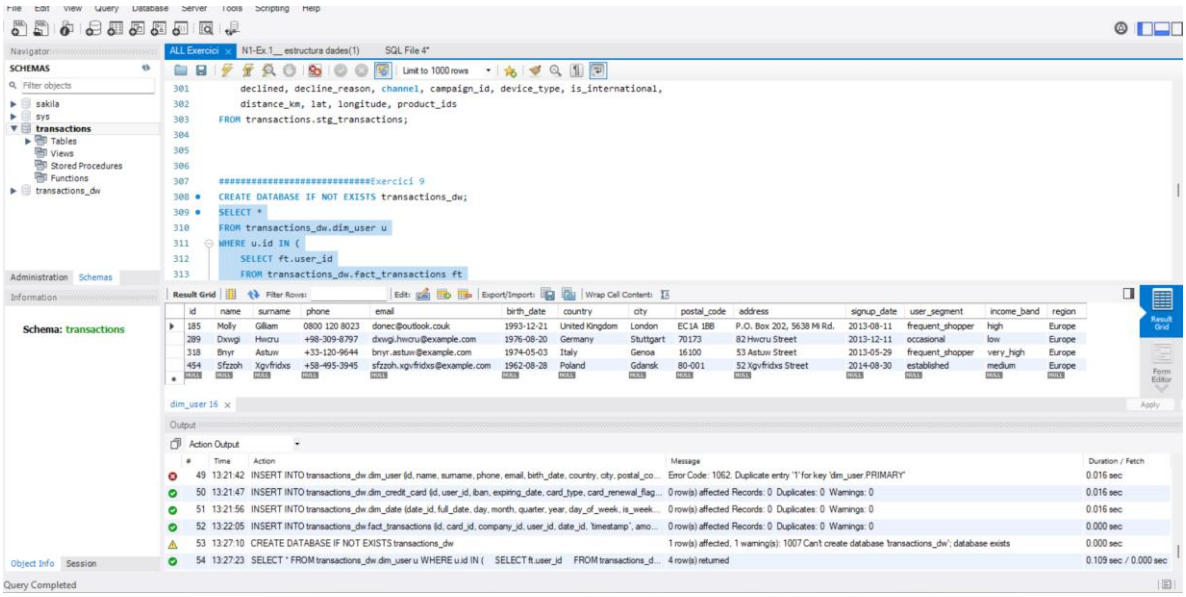
Tratamiento de product_ids

No se guarda product_ids como una cadena con varios valores en la tabla de hechos. Cuando se entregue products.csv, se añadirá dim_product y una tabla puente bridge_transaction_product con una fila por combinación
de transacción y producto.

Exercici 9

ejecución con la base de datos completa

Esta versión se ejecutó después de cargar por completo los datos reales (5.000 usuarios en dim_user y 100.000 transacciones en fact_transactions), obteniendo el resultado real de la consulta.

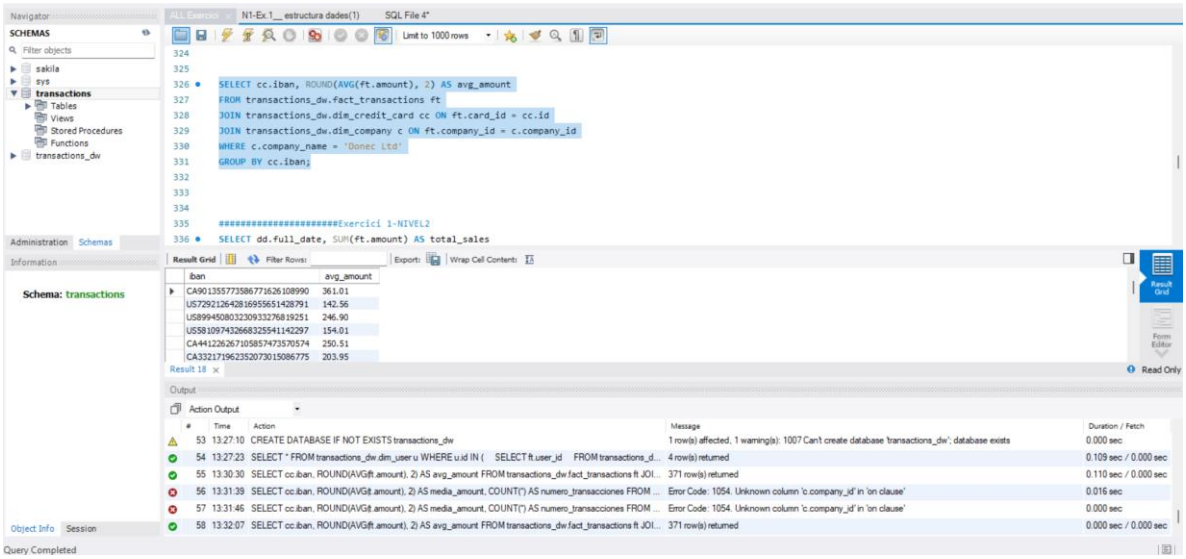


Exercici 10

La consulta relaciona las tarjetas con sus transacciones mediante card_id y las transacciones con la empresa mediante business_id. Filtra la empresa solicitada y calcula la media de amount para cada IBAN.

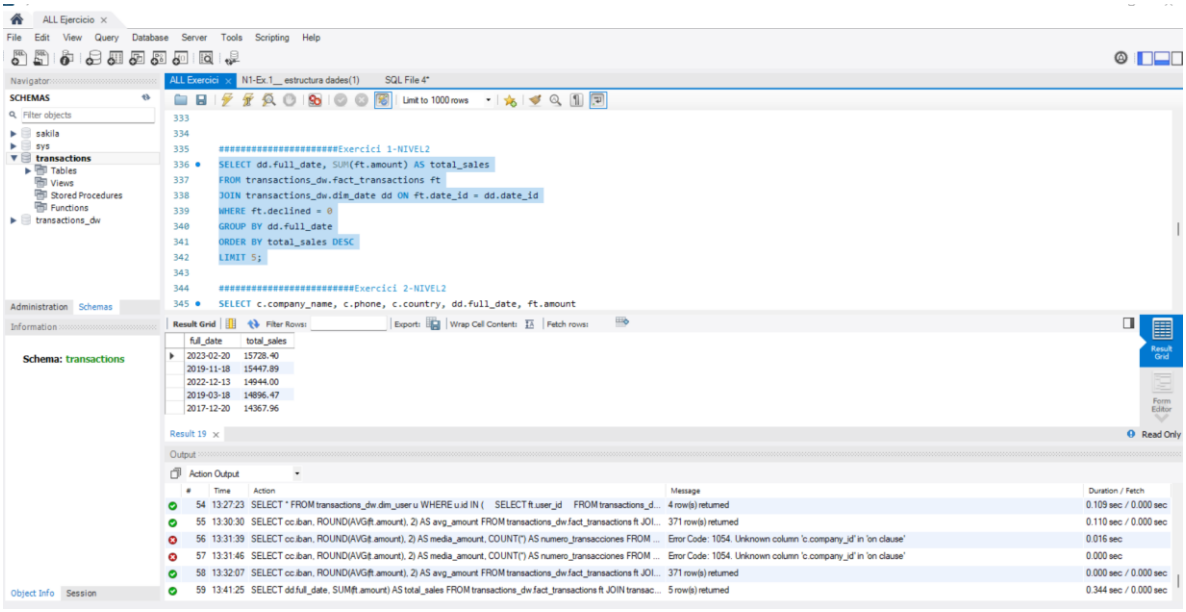
Resultado obtenido en los CSV

Para Donec Ltd se encontraron 449 transacciones asociadas a 371 IBAN distintos. La tabla muestra la media redondeada a dos decimales y el número de transacciones por tarjeta. Se incluyen todas las transacciones, sin filtrar por estado de rechazo.



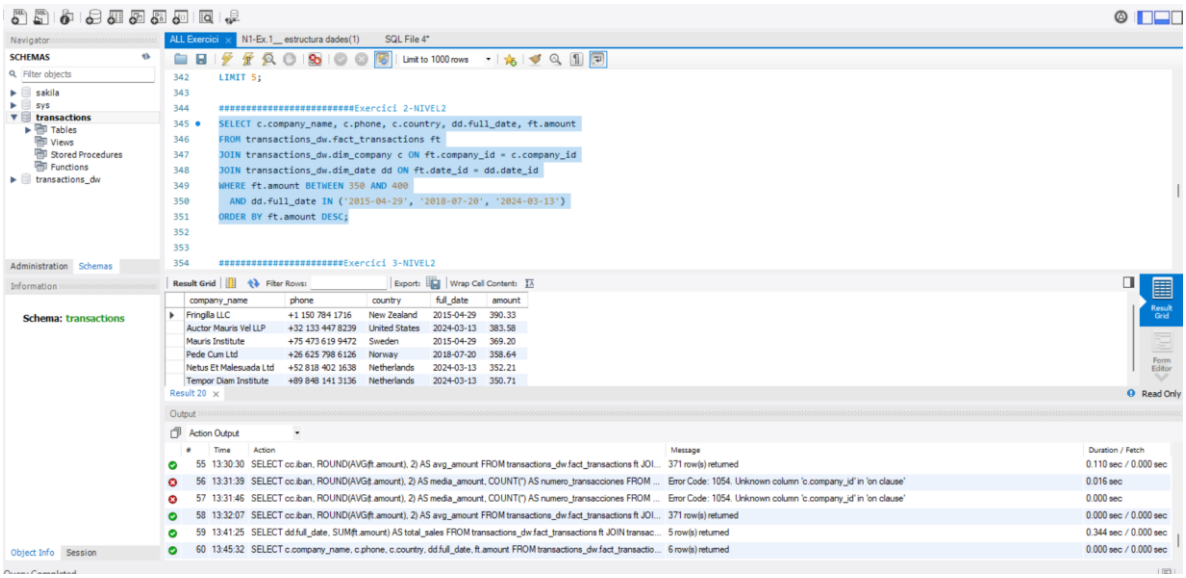
Nivel 2-Exercici 1

La consulta agrupa las transacciones por fecha y suma amount para obtener los ingresos diarios. El filtro declined = 0 excluye las operaciones rechazadas; finalmente, se ordenan los días de mayor a menor total y se muestran los cinco primeros



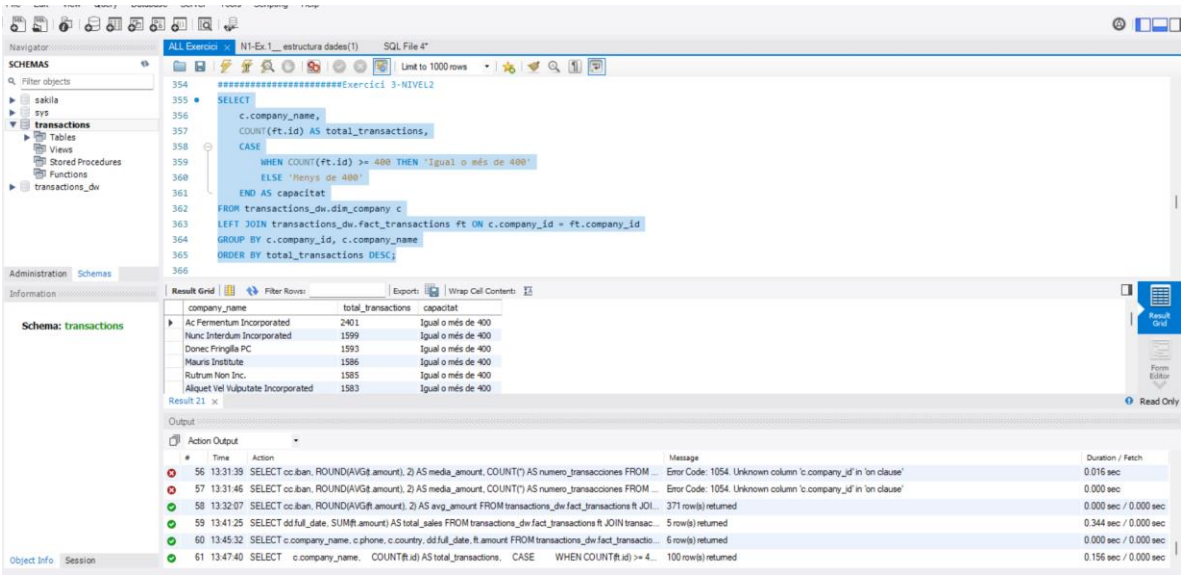
Nivel 2-Exercici 2

Se utiliza BETWEEN, que incluye los límites 350 y 400. Al no indicarse un filtro por estado, se consideran todas las transacciones coincidentes. Se encontraron 6 resultados, ordenados de mayor a menor importe.



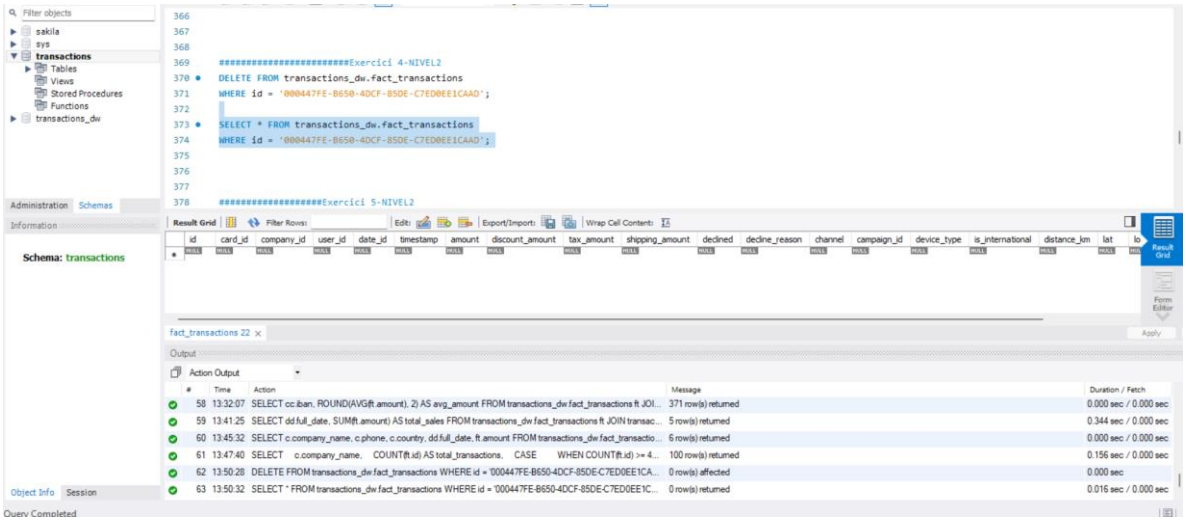
Nivel 2- Exercici 3

para optimizar la asignación de recursos, el departamento de recursos humanos necesita un listado de empresas que indique si el número de transacciones de cada una es igual o mayor a 400, o menor de 400.



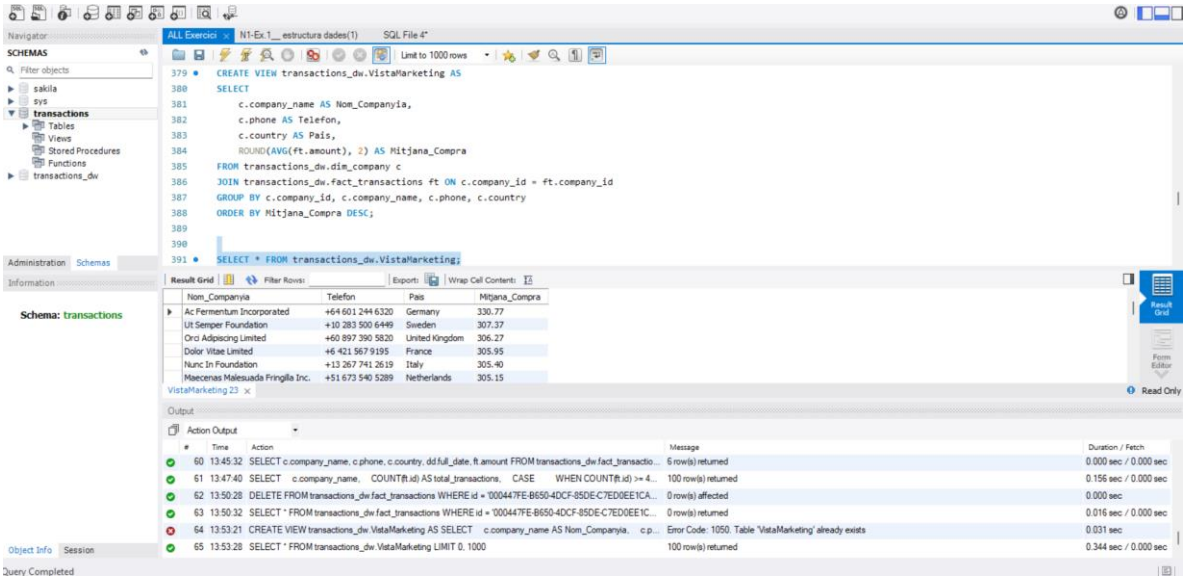
Nivel 2-Exercici 4

La primera consulta de verificación, ROW_COUNT(), debe indicar 1 si se eliminó el registro. Después de confirmar la transacción, el recuento de coincidencias restantes debe ser 0.



Nivel 2-Exercici 5

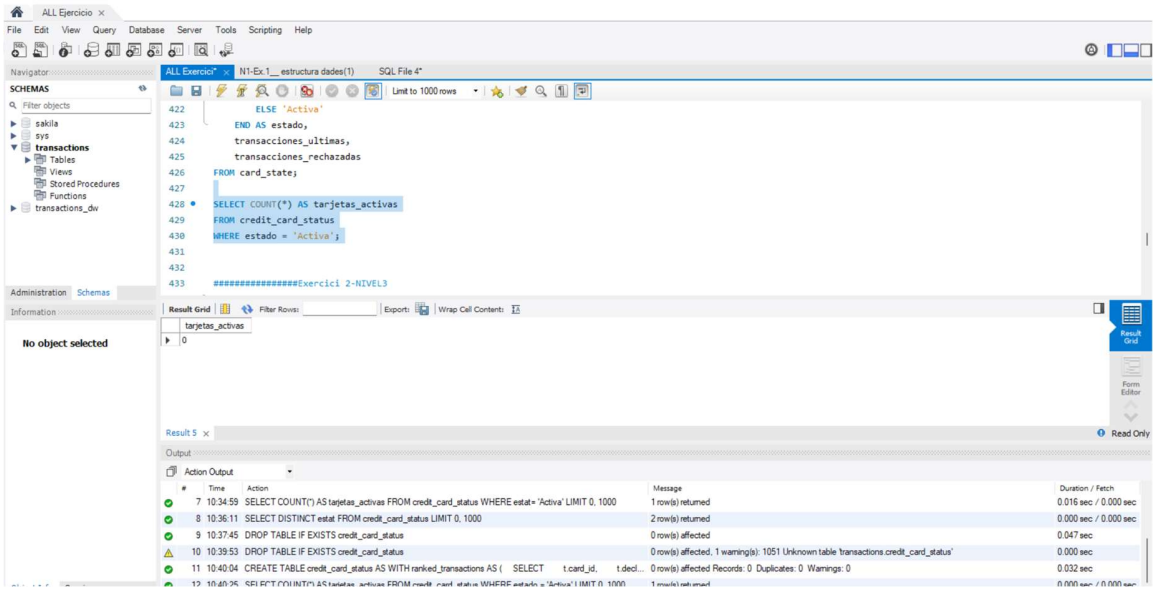
La vista agrupa las transacciones por compañía y calcula AVG(amount). La consulta final muestra las columnas solicitadas y ordena la media de mayor a menor. Se incluyen todas las transacciones, ya que no se pidió excluir las rechazadas. Resultado calculado con los CSV: 100 compañías.



Nivel 3 - Ejercicio 1

Estado de las tarjetas según sus tres transacciones más recientes

Se ordenan las transacciones de cada tarjeta por fecha descendente; el ID desempata fechas iguales. Si las tres últimas están rechazadas, el estado es **Inactiva**. Si hay al menos una aceptada, es **Activa**. Las tarjetas con menos de tres operaciones quedan como «Datos insuficientes»; las que no tienen ninguna, como «Sin transacciones».



Nivel 3 - Ejercicio 2

Modelo de datos

Como una transacción puede contener varios productos y un producto puede aparecer en muchas transacciones, se necesita una tabla intermedia. La tabla transaction_product descompone la lista separada por comas de transaction.product_ids en una fila por producto y transacción.



